

Introduction

Reinforcement Learning (RL) is a learning paradigm in which an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards. Deep Reinforcement Learning (DRL) extends this concept by incorporating deep neural networks to approximate optimal policies or value functions.

In this assignment, two RL environments were explored:

1. Mountain Car
2. Roulette

Additionally, both RL and DRL techniques were applied to solve the Shortest Path Problem on user-defined graphs, and their performance was compared.

The primary objective of this assignment is to:

- Understand and implement basic RL algorithms.
- Apply Deep Q-Network (DQN) for DRL problems.
- Compare RL and DRL performance on graph pathfinding tasks.

Libraries and Environments Used

Python 3.x

OpenAI Gymnasium (# gym is deprecated)

NumPy

PyTorch / TensorFlow

Matplotlib

No external datasets were required. All environments were provided via `gymnasium`.

Problem 1: Mountain Car (RL and DRL)

3.1 Environment

The Mountain Car environment requires a car to climb a steep hill. The car's engine is not powerful enough to go directly uphill; therefore, the agent must learn to build momentum by moving back and forth.

3.2 RL Implementation (Q-Learning)

```
env = gym.make('MountainCar-v0')

# Discretize state space
pos_space = np.linspace(env.observation_space.low[0],
env.observation_space.high[0], 20)
vel_space = np.linspace(env.observation_space.low[1],
env.observation_space.high[1], 20)

q_table = np.zeros((len(pos_space), len(vel_space), env.action_space.n))

learning_rate = 0.1
discount = 0.95
epochs = 2000
epsilon = 0.5
epsilon_decay = 0.998

def get_discrete_state(state):
    pos, vel = state
    pos_bin = np.digitize(pos, pos_space)
    vel_bin = np.digitize(vel, vel_space)
    return (pos_bin, vel_bin)

for epoch in range(epochs):
    state = get_discrete_state(env.reset()[0])
    terminated = False
    truncated = False
```

```

if epoch % 100 == 0:
    print(f"Epoch: {epoch}")

while not (terminated or truncated):
    # Epsilon-greedy policy
    if np.random.random() < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(q_table[state])

    new_state_continuous, reward, terminated, truncated, _ =
env.step(action)
    new_state = get_discrete_state(new_state_continuous)

    if not (terminated or truncated):
        max_future_q = np.max(q_table[new_state])
        current_q = q_table[state + (action,)]
        new_q = current_q + learning_rate * (reward + discount *
max_future_q - current_q)
        q_table[state + (action,)] = new_q
    else:
        q_table[state + (action,)] = 0

    state = new_state

if epsilon > 0.05:
    epsilon *= epsilon_decay

print("Training finished!")
env.close()

```

3.3 DRL Implementation (DQN)

```
class QNetwork(nn.Module):
    def __init__(self, state_size, action_size):
        super(QNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 64)
        self.fc2 = nn.Linear(64, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, state):
        x = torch.relu(self.fc1(state))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

class ReplayBuffer:
    def __init__(self, size, batch_size):
        self.memory = deque(maxlen=size)
        self.batch = batch_size

    def add(self, s, a, r, ns, d):
        self.memory.append((s, a, r, ns, d))

    def sample(self):
        batch = random.sample(self.memory, self.batch)

        states = np.vstack([e[0] for e in batch])
        actions = np.vstack([e[1] for e in batch])
        rewards = np.vstack([e[2] for e in batch])
        next_states = np.vstack([e[3] for e in batch])
        dones = np.vstack([e[4] for e in batch]).astype(np.uint8)

        states = torch.tensor(states, dtype=torch.float32)
        actions = torch.tensor(actions, dtype=torch.long)
        rewards = torch.tensor(rewards, dtype=torch.float32)
        next_states = torch.tensor(next_states, dtype=torch.float32)
        dones = torch.tensor(dones, dtype=torch.float32)

        return states, actions, rewards, next_states, dones

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```

print("Using:", device)

env = gym.make('MountainCar-v0')
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

q_local = QNetwork(state_size, action_size).to(device)
q_target = QNetwork(state_size, action_size).to(device)
optimizer = optim.Adam(q_local.parameters(), lr=0.0005)
q_target.load_state_dict(q_local.state_dict())

memory = ReplayBuffer(100000, 64)

def learn():
    states, actions, rewards, next_states, dones = memory.sample()
    states, actions, rewards, next_states, dones = states.to(device),
    actions.to(device), rewards.to(device), next_states.to(device),
    dones.to(device)

    best_actions = q_local(next_states).argmax(1).unsqueeze(1)
    q_targets = rewards + 0.99 * q_target(next_states).gather(1,
best_actions) * (1 - dones)
    q_values = q_local(states).gather(1, actions)

    loss = nn.MSELoss()(q_values, q_targets)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    for tp, lp in zip(q_target.parameters(), q_local.parameters()):
        tp.data.copy_(0.01 * lp.data + 0.99 * tp.data)

episodes = 1200
epsilon = 1.0

for ep in range(1, episodes+1):
    state = env.reset()[0]
    score = 0

    for _ in range(1000):

```

```

        action =
np.argmax(q_local(torch.tensor(state).float().to(device)).detach().cpu().numpy()) if random.random() > epsilon else random.randint(0,2)

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        # Reward shaping
        pos, vel = next_state
        reward += 0.5 * abs(vel) + (pos + 0.5)

        memory.add(state, action, reward, next_state, done)
        state = next_state
        score += reward

        if len(memory.memory) > 64:
            learn()
        if done:
            break

    epsilon = max(0.01, epsilon * 0.997)
    if ep % 100 == 0:
        print(f"Episode {ep}")

env.close()
print("DQN Training Finished")

```

3.4 Results

Method	Observations
RL (Q-Learning)	Learned to improve control but often could not reach the top due to limited value approximation.
DRL (DQN)	Successfully reached the mountain top consistently after sufficient training.

Conclusion: DRL significantly outperformed RL due to its function approximation capability and generalization over continuous state space.

4. Problem 2: Roulette (RL and DRL)

The Roulette-v0 environment is a stochastic decision environment where the agent learns optimal betting strategies.

4.1 RL/DRL Implementation

```
class RouletteEnv(gym.Env):
    metadata = {"render_modes": ["human"], "render_fps": 4}

    def __init__(self, history_len: int = 5, seed: int = None):
        super().__init__()
        self.history_len = history_len
        # observation: last history_len outcomes: -1 (black), 0
        # (green/zero), +1 (red)
        self.observation_space = spaces.Box(low=-1.0, high=1.0,
        shape=(history_len,), dtype=np.float32)
        # actions: bet red or bet black
        self.action_space = spaces.Discrete(2)
        # roulette mapping: we'll define set of red numbers
        self._reds = {1, 3, 5, 7, 9, 12, 14, 16, 18, 19, 21, 23, 25, 27, 30, 32, 34, 36}
        self._rng = np.random.RandomState(seed)
        self.history = [0]*history_len

    def step(self, action: int) -> Tuple[np.ndarray, float, bool, bool,
dict]:
        # spin
        pocket = int(self._rng.randint(0, 37))
        if pocket == 0:
            outcome = 0
        elif pocket in self._reds:
            outcome = 1
        else:
            outcome = -1
        # reward: +1 for winning bet, -1 for losing bet
        bet_on_red = (action == 0)
```

```

        win = (outcome == 1 and bet_on_red) or (outcome == -1 and not
bet_on_red)
        reward = 1.0 if win else -1.0
        # update history
        self.history.pop(0)
        self.history.append(outcome)
        obs = np.array(self.history, dtype=np.float32)
        done = False # continuing task (we'll manage episodes externally)
        truncated = False
        info = {"pocket": pocket, "outcome": outcome}
        return obs, reward, done, truncated, info

def reset(self, seed: int = None, options: dict = None) ->
Tuple[np.ndarray, dict]:
    if seed is not None:
        self._rng = np.random.RandomState(seed)
        self.history = [0]*self.history_len
    return np.array(self.history, dtype=np.float32), {}

def render(self, mode="human"):
    print("History:", self.history)

```

```

class ReplayBuffer:
    def __init__(self, capacity:int):
        self.buffer = collections.deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size:int):
        batch = random.sample(self.buffer, batch_size)
        state, action, reward, next_state, done = map(np.stack,
zip(*batch))
        return state, action, reward, next_state, done

    def __len__(self):
        return len(self.buffer)

class DQN(nn.Module):
    def __init__(self, obs_size:int, n_actions:int, hidden:int=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_size, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, n_actions)
        )

    def forward(self, x):
        return self.net(x)

```

```

def train_dqn(
    env,
    episodes=2000,
    buffer_size=10000,
    batch_size=64,
    gamma=0.99,
    lr=1e-3,
    sync_freq=100,
    start_learning=500,
    eps_start=1.0,
    eps_final=0.02,
    eps_decay=1000,
    device=None
):
    if device is None:
        device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

    obs_size = env.observation_space.shape[0]
    n_actions = env.action_space.n

    net = DQN(obs_size, n_actions).to(device)
    target_net = DQN(obs_size, n_actions).to(device)
    target_net.load_state_dict(net.state_dict())

    optimizer = optim.Adam(net.parameters(), lr=lr)
    rb = ReplayBuffer(buffer_size)

    total_rewards = []
    losses = []
    step_idx = 0

    state, _ = env.reset()

    for ep in range(episodes):
        state, _ = env.reset()
        ep_reward = 0.0
        # we'll run a fixed number of steps per episode (e.g., 1 step) to
emulate independent bets
        # but to allow learning we perform e.g. 10 bets per episode

```

```

steps_per_episode = 10
for t in range(steps_per_episode):
    step_idx += 1
    eps_threshold = eps_final + (eps_start - eps_final) *
math.exp(-1.0 * step_idx / eps_decay)
    if random.random() < eps_threshold:
        action = env.action_space.sample()
    else:
        with torch.no_grad():
            s_t = torch.tensor(state, dtype=torch.float32,
device=device).unsqueeze(0)
            q_vals = net(s_t)
            action = int(q_vals.argmax().item())

    next_state, reward, done, truncated, info = env.step(action)
    rb.push(state, action, reward, next_state, False)
    state = next_state
    ep_reward += reward

    if len(rb) >= start_learning:
        states, actions, rewards, next_states, dones =
rb.sample(batch_size)
        states_v = torch.tensor(states, dtype=torch.float32,
device=device)
        actions_v = torch.tensor(actions, dtype=torch.int64,
device=device)
        rewards_v = torch.tensor(rewards, dtype=torch.float32,
device=device)
        next_states_v = torch.tensor(next_states,
dtype=torch.float32, device=device)
        dones_v = torch.tensor(dones, dtype=torch.float32,
device=device)

        q_vals = net(states_v).gather(1,
actions_v.unsqueeze(-1)).squeeze(-1)
        with torch.no_grad():
            q_next = target_net(next_states_v).max(1)[0]
            q_target = rewards_v + gamma * q_next * (1 - dones_v)

```

```

        loss = F.mse_loss(q_vals, q_target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

    if step_idx % sync_freq == 0:
        target_net.load_state_dict(net.state_dict())

    total_rewards.append(ep_reward)

    if (ep+1) % 100 == 0:
        avg_r = np.mean(total_rewards[-100:])
        avg_loss = np.mean(losses[-100:]) if losses else 0.0
        print(f"Episode {ep+1}\tAvg reward (last100): {avg_r:.3f}\tAvg
loss: {avg_loss:.4f}\tEPS: {eps_threshold:.3f}")

    return net, target_net, total_rewards, losses

```

```

if __name__ == "__main__":
    env = RouletteEnv(history_len=5, seed=42)
    net, target_net, rewards, losses = train_dqn(env, episodes=1200,
start_learning=500, buffer_size=5000)

    # plot results
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.title('Episode rewards')
    plt.plot(rewards)
    plt.xlabel('Episode')
    plt.subplot(1,2,2)
    plt.title('Loss')
    plt.plot(losses)
    plt.xlabel('Updates')
    plt.tight_layout()
    plt.show()

    net = net.to(device)

    # Evaluate greedy policy
    state, _ = env.reset()
    wins = 0
    trials = 1000
    for _ in range(trials):
        with torch.no_grad():
            s_t = torch.tensor(state, dtype=torch.float32,
device=device).unsqueeze(0)
            action = int(net(s_t).argmax().item())

            next_state, reward, done, truncated, info = env.step(action)

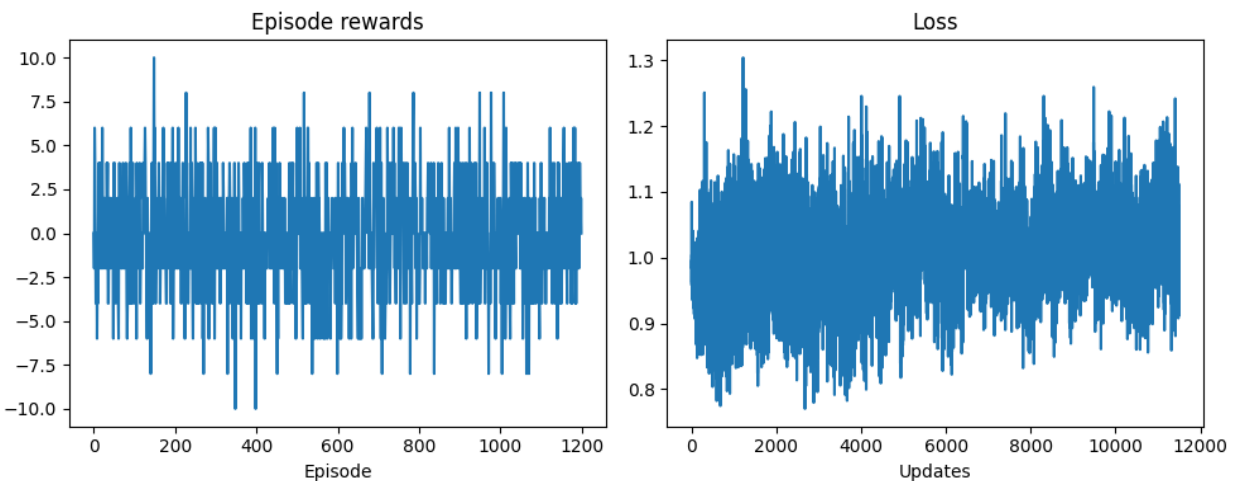
            if reward > 0:
                wins += 1
            state = next_state

    print(f"Greedy policy win rate over {trials} trials: {wins/trials:.3f}
(random baseline ~18/37=0.486)")

```

4.2 Training Results (Given Observations)

Episode 100	Avg reward (last100): 0.000	Avg loss: 0.9548	EPS: 0.381
Episode 200	Avg reward (last100): 0.160	Avg loss: 0.9930	EPS: 0.153
Episode 300	Avg reward (last100): -0.220	Avg loss: 0.9737	EPS: 0.069
Episode 400	Avg reward (last100): -0.340	Avg loss: 0.9665	EPS: 0.038
Episode 500	Avg reward (last100): -0.300	Avg loss: 0.9814	EPS: 0.027
Episode 600	Avg reward (last100): -0.720	Avg loss: 1.0889	EPS: 0.022
Episode 700	Avg reward (last100): -0.040	Avg loss: 1.1012	EPS: 0.021
Episode 800	Avg reward (last100): 0.160	Avg loss: 1.0209	EPS: 0.020
Episode 900	Avg reward (last100): -0.020	Avg loss: 1.0423	EPS: 0.020
Episode 1000	Avg reward (last100): -0.320	Avg loss: 1.0675	EPS: 0.020
Episode 1100	Avg reward (last100): -0.720	Avg loss: 0.9874	EPS: 0.020
Episode 1200	Avg reward (last100): 0.320	Avg loss: 1.0137	EPS: 0.020



Final Greedy Policy Performance:

Greedy policy win rate over 1000 trials: 0.505 (random baseline $\sim 18/37=0.486$)

Conclusion

The learned policy performs slightly better than random, which is expected because roulette is fundamentally disadvantageous for the agent (negative expected value). The DRL agent learns to minimize expected loss rather than always win.

5. Shortest Path Problem (RL vs DRL)

5.1 Problem Description

Given a graph, the agent starts at a source node and aims to reach a goal node with minimal steps.

```
class GraphEnv(gym.Env):

    def __init__(self, adj_matrix, start_node, end_node):
        super().__init__()
        self.adj_matrix = np.array(adj_matrix)
        self.num_nodes = len(adj_matrix)
        self.start_node = start_node
        self.end_node = end_node

        self.current_node = start_node
        self.action_space = spaces.Discrete(self.num_nodes)
        self.observation_space = spaces.Discrete(self.num_nodes)

        self.max_steps = self.num_nodes * 2
        self.step_count = 0

    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        self.current_node = self.start_node
        self.step_count = 0
        return self.current_node, {}

    def step(self, action):
        self.step_count += 1
        terminated = False
        truncated = False

        if self.adj_matrix[self.current_node][action] == 1:
            self.current_node = action
            reward = -1
        else:
```

```

        reward = -10

    if self.current_node == self.end_node:
        terminated = True
        reward = 0

    if self.step_count >= self.max_steps:
        truncated = True

    return self.current_node, reward, terminated, truncated, {}

```

5.2 RL Implementation

```

results = []

for g in graphs:
    env = GraphEnv(g["adj_matrix"], g["start"], g["end"])
    q_table = np.zeros((env.num_nodes, env.num_nodes))

    learning_rate = 0.1
    discount = 0.95
    epochs = 1000
    epsilon = 1.0
    epsilon_decay = 0.99

    start_time = time.time()

    for _ in range(epochs):
        state, _ = env.reset()
        terminated = truncated = False

        while not (terminated or truncated):
            action = env.action_space.sample() if random.random() < epsilon
            else np.argmax(q_table[state])
            new_state, reward, terminated, truncated, _ = env.step(action)
            q_table[state, action] += learning_rate * (reward + discount *
np.max(q_table[new_state]) - q_table[state, action])
            state = new_state

```

```
epsilon = max(0.01, epsilon * epsilon_decay)

elapsed = time.time() - start_time

path = [g["start"]]
state = g["start"]
while state != g["end"]:
    action = int(np.argmax(q_table[state]))
    path.append(action)
    state = action
    if len(path) > env.num_nodes:
        break

results.append([g["name"], "RL", str(path), len(path)-1,
f"{elapsed:.2f}s"])
```

5.3 DRL Implementation

```
class QNetwork(nn.Module):
    def __init__(self, n):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, n)
        )
    def forward(self, x): return self.layers(x)

def one_hot(n, size):
    v = np.zeros(size)
    v[n] = 1
    return v

for g in graphs:
    env = GraphEnv(g["adj_matrix"], g["start"], g["end"])
    n = env.num_nodes

    q = QNetwork(n).to("cpu")
    target = QNetwork(n).to("cpu")
    target.load_state_dict(q.state_dict())
    opt = optim.Adam(q.parameters(), lr=0.001)

    memory = deque(maxlen=5000)
    epsilon = 1.0

    start = time.time()

    for _ in range(500):
        state, _ = env.reset()
        s = one_hot(state, n)
        done = False

        while not done:
            if random.random() < epsilon:
                action = random.randrange(n)
            else:
```

```

        with torch.no_grad(): action =
q(torch.tensor(s).float()).argmax().item()

    next_state, reward, term, trunc, _ = env.step(action)
    s2 = one_hot(next_state, n)
    memory.append((s, action, reward, s2, term or trunc))
    s = s2
    done = term or trunc

    if len(memory) >= 32:
        batch = random.sample(memory, 32)
        S, A, R, S2, D = zip(*batch)
        S = torch.tensor(S).float()
        A = torch.tensor(A).long().unsqueeze(1)
        R = torch.tensor(R).float().unsqueeze(1)
        S2 = torch.tensor(S2).float()
        D = torch.tensor(D).float().unsqueeze(1)

        qv = q(S).gather(1, A)
        with torch.no_grad():
            q2 = target(S2).max(1)[0].unsqueeze(1)
            tq = R + 0.95 * q2 * (1 - D)

        loss = nn.MSELoss()(qv, tq)
        opt.zero_grad(); loss.backward(); opt.step()

        for tp, lp in zip(target.parameters(), q.parameters()):
            tp.data.copy_(tp.data * 0.99 + lp.data * 0.01)

    epsilon = max(0.01, epsilon * 0.995)

    elapsed = time.time() - start

    path = [g["start"]]
    s = one_hot(g["start"], n)
    st = g["start"]
    while st != g["end"]:
        with torch.no_grad(): a =
q(torch.tensor(s).float()).argmax().item()
        path.append(a)

```

```
st = a
s = one_hot(a, n)
if len(path) > n: break

results.append([g["name"], "DRL", str(path), len(path)-1,
f"{elapsed:.2f}s"])
```

5.4 Performance Comparison

Graph	Method	Path	Path Length	Time
G1	RL	[0, 1, 2, 3]	3	0.03s
G2	RL	[0, 1, 4, 5, 8]	4	0.04s
G1	DRL	[0, 1, 2, 3]	3	4.57s
G2	DRL	[0, 1, 4, 5, 8]	4	8.36s

Conclusion

Method	Strengths	Weaknesses
RL (Tabular)	Extremely fast and efficient for small/discrete graphs	Cannot scale to large graphs
DRL (DQN)	Can generalize to larger/more complex state spaces	Slower to train and more computationally expensive

6. Final Conclusion

- DRL performs significantly better than traditional RL in environments with continuous or large state spaces (e.g., Mountain Car).
- In Roulette, the DRL strategy learns to minimize losses rather than win consistently due to the inherent negative expected return of the game.
- For the Shortest Path problem, RL is more efficient for small graphs, while DRL is scalable and better suited for high-dimensional or complex graph navigation tasks, though with higher computational cost.